Instituto Superior de Formación Técnica Nº 151
Carrera: Analista en Sistemas
3 Año. Algoritmos y Estructuras de Datos III.
Alumna: LUZURIAGA, Silvana

LISTAS – LISTAS ENLAZADAS DOBLES y LISTAS STL.

Marco Teórico:

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1. Describir el Concepto de Lista
Una lista es una colección de elementos (o nodos) organizados de manera lineal, donde cada elemento almacena un dato o informacion relevante. A diferencia de los arreglos estaticos, las listasa enlazadas son estrucuturas de datos dinamicas, lo que significa que su tamaño puede crecer o achicarse en tiempo de ejecucion de acuerdo a las necesidades del programa, reservando memoria de manera dispersa y flexible.

2. Describir le Concepto de:

2.1 **Lista Vinculada**: es una lista en la que cada nodo contiene dos partes: **el dato** propiamente dicho y **un puntero (enlace)**que apunta hacia el siguiente nodo de la lista. El ultimo nodo de la estructura apunta a **NULL** (o un valor vacío), lo que marca el final de la lista. Su recorrido es unidireccional (solo se puede avanzar hacia adelante)

2.2 **Lista Doblemente Vinculada:** es una estructura mas avanzada en la cual cada nodo posee tres partes: **el dato**, **un puntero al siguiente nodo y un puntero adicional al nodo anterior.** Esto permite recorrer la lista de manera bidireccional (hacia adelanta y hacia atrás), facilitando operaciones de borrado o insercion en posiciones intermedias a cambio de un pequeño costo extra en memoria.

2.3 **Lista Circular:** es una variante de la lista enlazada simple con la particularidad de que el ultimo nodo, en lugar de apuntar a **NULL**, **apunta directamente de regreso al primero nodo (cabeza)** de la lista. De esta manera, se forma un ciclo o anillo continuo que permite recorrer toda la estructura de forma infinita sin encontrar un final absoluto.

2.4 **Lista Circular doblemente Vinculada:** combina las características de las dos anteriores. Cada nodo tiene enlaces a su predecesor y a su sucesor. La gran diferencia es la conexión **de los extremos: el campo siguiente del ultimo nodo apunta al primero, y el campo anterior del primer nodo apunta al ultimo**, cerrando un **circuito cerrado bidireccional** perfecto.

3. Que beneficios tiene usar <LIST> de la STL?
La STL (Standard Template Library) de C++ provee el contenedor nativo **#include <list>,** el cual implementa internamente una **lista doblemente enlazada**. Sus principales beneficios son:
*** Inserciones y eliminaciones ultra rápidas**: (*O(1)):* agregar o quitar elementos en cualquier posicion de la

lista(especialmente en los extremos) es sumamente eficiente, ya que solo se deben reacomodar punteros sin necesidad de desplazar el resto de los elementos en memoria.
***Gestión dinámica y automática**: no requiere preocuparse por la reserva o liberacion manual de memoria (new/delete), ya que la STL se encarga por completo del ciclo de vida de los nodos.
***Iteradores integrados**: permite navegar de manera estandarizada y segura por toda la estructura utilizando punteros inteligentes (iterators), compatibles con todos los algoritmos genéricos de C++

4. Enumerar y Describir las Operaciones **básicas** que Implementan las Listas.
Las operaciones esenciales para la manipulacion del listas son:
***Acceso/Lectura(Access):** permite consultar el valor de un nodo en una posicion dada. En lista, requiere recorrer secuencialmente la estructura desde el inicio hasta el nodo deseado (*O(n))*
***Busqueda(Search):** recorre la lista comparando un valor clave elemento por elemento para determinar si existe en la lista y devolver su posicion. Su peor caso es de orden lineal (*O(n)).*
***Insercion (Insertion):** consiste en crear un nuevo nodo, asignarle un dato y enlazarlo correctamente en la estructura. Puede realizarse al principio (cabeza), al final (cola) o en una posicion intermedia.
*Eliminacion / Borrado(Deletion): remueve un nodo especifico de la lista, liberando la memoria asignada y reconfigurando los punteros de los nodos vecinos para no romper la cadena de enlaces.

5. Dar 2 formas de Manejo de Error a listas vacías?
Cuando se intenta realizar una operación invalida en una lista que no contiene elementos (como intentar borrar un nodo o leer la cabeza), el sistema puede fallar catastróficamente. Dos formas estándar de manejar este error son:
A- **Validacion previa mediante Condicionales de Control**: antes de ejecutar cualquier extracción o lectura, se utiliza un metodo de control (como **.empty()** de la STL o verificar si (**cabeza== Null)** dentro de una estructura **if**. Si da verdadero, se aborta la operación imprimiendo un mensaje controlado al usuario (ej: “Error: la lista esta vacia, no se pueden remover elementos”).
B- Lanzamiento y captura de Excepciones (Try – Catch): consiste en delegar la seguridad al bloque de excepciones del lenguaje. Si la funcion detecta que la lista esta vacia al intentar acceder a un dato, lanza un error controlado **(Throw out_of_range(“Lista vacia”))**. Esto interrumpe la ejecución lineal y redirige el flujo hacia el bloque **catch**, evitando que la aplicación crashee de forma imprevista.

Marco Practico:

1. Un Clínica Local nos pide Implementar un Sistema para manejar Pacientes.
   Debe tener un menú que permita implementar una Lista que facilite: registrar, modificar, eliminar y listar pacientes.
   En este enfoque, construimos la lista de manera nativa estructurando los nodos y manipulando los enlaces a mano

2. Idem 1 pero Implementando Listas en STL
   Aqui resolvemos el mismo problema, pero delegando toda la complejidad de punteros, creacion de nodos y liberacion de memoria al contenedor nativos de C++.

   C**omparto el link del repo:**

https://github.com/Sylvana74/Algoritmos_Estructuras_De_Datos_3_2026/tree/main/UNIDAD%203